

Code-Layout, Readability and Reusability

Date	01 October 2026
Team ID	SWTID-2026-9629
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the structure, readability and reusability of the PocketSmart AI codebase (Flask application in the PocketSmart-AI folder).

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Python files use 4-space indentation; HTML templates, CSS and JavaScript use 2 spaces consistently.
2	Proper File Structure	Files and folders are logically organized	Yes	app/ holds __init__.py, auth.py, main.py, ai.py, models.py, templates/ and static/; config.py, run.py, requirements.txt and .env.example sit at the root.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as planner_type, payload, image_path, allocated_total, remaining_budget and result_json describe their content.
4	Function / Method Names	Functions are descriptively named	Yes	generate_recommendation(), platform_url(), allowed_image(), clean_number(), set_password() and check_password() state what they do.
5	Code Comments	Inline and block comments explain logic	Partial	The code has one inline comment and no docstrings; the Gemini prompt and the README (setup, testing, troubleshooting) explain the behaviour. More docstrings are planned.
6	Modular Design	Code is split into reusable functions/modules	Yes	Blueprints separate auth and main routes; Gemini logic is isolated in ai.py; models, configuration and templates are separate modules.
7	No Redundant Code	Duplicate or unused code is removed	Partial	One planner() route serves all three planners and one AI function serves all prompts; the three planner templates repeat the result-placeholder block.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	planner() wraps processing in try/except, rolls back the session and flashes the message; ai.py raises clear errors for a missing API key and invalid JSON.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	generate_recommendation() in app/ai.py (prompt, Gemini call, JSON parsing)	Python, google-genai	Home, Party and Jewelry planners	High
2	planner() route in app/main.py	Python, Flask	/planner/home, /planner/party, /planner/jewelry	High
3	base.html layout with navigation and footer	HTML, Jinja2, CSS	Every page of the application	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
4	_result.html result partial	HTML, Jinja2	Three planner pages and the saved-plan page (history_detail.html)	High
5	Helpers platform_url(), clean_number(), allowed_image()	Python	All planner requests	Medium
6	Config class in config.py (reads .env values)	Python	Whole application (database, Gemini key and model, upload rules)	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation of routes, AI logic, models, configuration, templates and static files.
Readability	4	Short functions with descriptive names; the planner() route is long but easy to follow.
Reusability	4	One AI function, one planner route and shared templates serve all three planners.
Documentation / Comments	3	README is detailed; inline comments and docstrings are limited.
Overall Score	4	Well-structured MVP; adding docstrings and tests would raise it further.